
Algorithm 2: Tabular policy evaluation using TD-style update

Require: Number of particles N , kernel k , discount factor $\gamma \in [0, 1]$, evaluation policy π , learning rate α_t , tabular particles $\{\theta_i(s, a)\}_{i=1}^N$

Initialization: initial particles θ , initial copy particles $\theta^- \leftarrow \theta$, and initial state s_0

1 **for** $t=1, 2, \dots$ **do**

2 Take action $a_t = \pi(s_t)$ and observe $s_{t+1} \sim P(\cdot | s_t, a_t)$ and $r_t \sim \mathcal{R}(s_t, a_t)$

3 Compute Bellman target particles

$$\hat{T}\theta_i^- \leftarrow r_t + \gamma\theta_i^-(s_{t+1}, \pi(s_{t+1})), \forall i \in \{1, \dots, N\}$$

4 Compute TD gradient

$$\textbf{MMDRL Case: } g_i \leftarrow \frac{\partial}{\partial \theta_i(s_t, a_t)} \text{MMD}_b^2(\{\theta_i(s_t, a_t)\}_{i=1}^N, \{\hat{T}\theta_i^-\}_{i=1}^N; k), \forall i \in \{1, \dots, N\}$$

$$\textbf{QRDRL Case: } g_i \leftarrow \frac{\partial}{\partial \theta_i(s_t, a_t)} \frac{1}{N} \sum_{j=1}^N \left(\hat{T}\theta_j^- - \theta_i(s_t, a_t) \right) \left(\frac{2i-1}{2N} - 1_{\{\hat{T}\theta_j^- < \theta_i(s_t, a_t)\}} \right), \forall i \in \{1, \dots, N\}$$

5 Update

$$\begin{aligned} \theta_i(s_t, a_t) &\leftarrow \theta_i(s_t, a_t) - \alpha_t g_i, \forall i \in \{1, \dots, N\} \\ \theta^- &\leftarrow \theta \end{aligned}$$

6 **end**

Output: Approximate distribution $\mu(s, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(s, a)}$

Atari game experiment

We extend MMDRL to DQN-like architecture to create a novel deep RL, namely MMDQN. We use the same architecture of DQN except that we change the last layer to the size of $N \times |\mathcal{A}|$, instead of the size $|\mathcal{A}|$. In addition, we replace the squared loss in DQN by the empirical MMD loss. The full details for MMDQN are provided in Algorithm 3. We expect that our framework would also benefit from recent orthogonal improvements to DQN such as double-DQN (van Hasselt, Guez, and Silver 2016), the dueling architecture (Wang et al. 2016) and prioritized replay (Schaul et al. 2016) but did not include these for simplicity. In Table 4, we provide the hyperparameter details of QR-DQN and MMDQN used in the Atari games. The hyperparameters in MMDQN that share with QR-DQN are intentionally set the same to allow for fair comparison.

Hyperparameters	QR-DQN	MMDQN
Learning rate	0.00005	0.00005
Optimizer	Adam	Adam
ϵ_{ADAM}	0.0003125	0.0003125
N	200	200
Quantiles	$\{\frac{2i-1}{2N} : 1 \leq i \leq N\}$	N/A
Kernel bandwidth	N/A	$\{1, 2, \dots, 9, 10\}$

Table 4: The MMDQN hyperparameters as compared to those of QR-DQN.

We evaluated our algorithm on 55 Atari 2600 games (Bellemare et al. 2013) following the standard training and evaluation procedures (Mnih et al. 2015; van Hasselt, Guez, and Silver 2016). For every 1M training steps in the environment, we computed the average scores of the agent by freezing the learning and evaluating the latest agent for 500K frames. We truncated episodes at 108K frames (equivalent to 30 minutes of game playing). We used the 30 no-op evaluation settings where we play a random number (up to 30) of no-op actions at the beginning of each episode during evaluation. We report the best score for a game by an algorithm which is the algorithm’s highest evaluation score in that game across all evaluation iterations during the training course (given the same hyperparameters are shared for all games).

The human normalized scores of an agent per game is the agent’s normalized scores such that 0% corresponds to a random agent and 100% corresponds to the average score of a human expert. The human-normalized scores used in the paper are

Algorithm 3: MMDQN

Require: Number of particles N , kernel k (e.g., Gaussian kernel), discount factor $\gamma \in [0, 1]$, learning rate α , replay buffer $\mathcal{M}$, main network Z_θ , target network Z_{θ^-} , and a policy π (e.g., ϵ -greedy policy w.r.t.

$$Q_\theta(s, a) = \frac{1}{N} \sum_{i=1}^N Z_\theta(s, a)_i, \forall s, a$$

1 Initialize θ and $\theta^- \leftarrow \theta$

2 **for** $t = 1, 2, \dots$ **do**

3 Take action $a_t \sim \pi(\cdot | s_t; \theta)$, receive reward $r_t \sim \mathcal{R}(\cdot | s_t, a_t)$, and observe $s_{t+1} \sim P(\cdot | s_t, a_t)$

4 Store (s_t, a_t, r_t, s_{t+1}) to the replay buffer $\mathcal{M}$

5 Randomly draw a batch of transition samples (s, a, r, s') from the replay buffer $\mathcal{M}$

6 Compute a greedy action

$$a^* \leftarrow \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{i=1}^N Z_{\theta^-}(s', a')_i$$

7 Compute the empirical Bellman target measure

$$\hat{T}Z_i^- \leftarrow r + \gamma Z_{\theta^-}(s', a^*)_i, \forall i \in \{1, \dots, N\}$$

8 Update the main network

$$\theta \leftarrow \theta - \alpha \Delta_\theta \text{MMD}_b \left(\{Z_\theta(s, a)_i\}_{i=1}^N, \{\hat{T}Z_i^-\}_{i=1}^N; k \right)$$

9 Periodically update the target network $\theta^- \leftarrow \theta$

10 **end**

explicitly defined by

$$\text{score} = \frac{\text{agent} - \text{random}}{\text{human} - \text{random}},$$

where *agent*, *human*, *random* denotes the raw scores (undiscounted returns) for the given agent, the reference human player and the random player (Mnih et al. 2015), resp., in each game.

The per-game percentage improvement of MMDQN over QR-DQN is computed as follow

$$\text{percentage_improvement} = \frac{\text{score}_{\text{MMDQN}} - \text{score}_{\text{QR-DQN}}}{\text{score}_{\text{QR-DQN}}} \times 100\%,$$

where $\text{score}_{\text{MMDQN}}$ and $\text{score}_{\text{QR-DQN}}$ are the best raw evaluation score of MMDQN and QR-DQN in the considered game. The log-scaled percentage improvement is computed as

$$\log_percentage_improvement = 1_{\{\text{percentage_improvement} \geq 0\}} \log(|\text{percentage_improvement}| + 1).$$

Appendix C. Further experimental results

In Figure 6 we visualize the behaviour of our MMDQN in the Breakout game. Three rows correspond to 3 consecutive frames of the Breakout game accompanied by the approximate return distributions learnt by MMDQN. Since the particles learnt by MMDQN represent empirical samples of the return distributions, we can visualize the return distributions via the learnt particles by plotting the histogram (with 17 bins in this example) of these particles. The learnt particles in MMDQN can maintain diversity in approximating the return distributions even though there is no order statistics in MMDQN as in the existing distributional RL methods such as QR-DQN. The 3 consecutive frames illustrate that the ball is moving away from the left to the right. In response, MMDQN also moves the paddle away from the left by gradually placing the probability mass of the return for the LEFT action towards smaller values. In particular, in the first frame where the ball is still far away from the ground, the MMDQN agent does not make a significant difference between actions. As the ball is moving closer to the ground from the left (the second and third frame), the agent becomes clearer that the LEFT action is not beneficial, thus placing the action’s probability mass to smaller values.

We also include videos of the moves and approximate return distributions learnt by MMDQN in the supplementary. The exact video addresses in the supplementary are shown in Table 5.

We show the median and mean of the test human-normalized scores across all 55 Atari games in Figure 7, the online learning curves in all the 55 Atari games in Figure 8 and provide the full raw scores of MMDQN in Table 6.

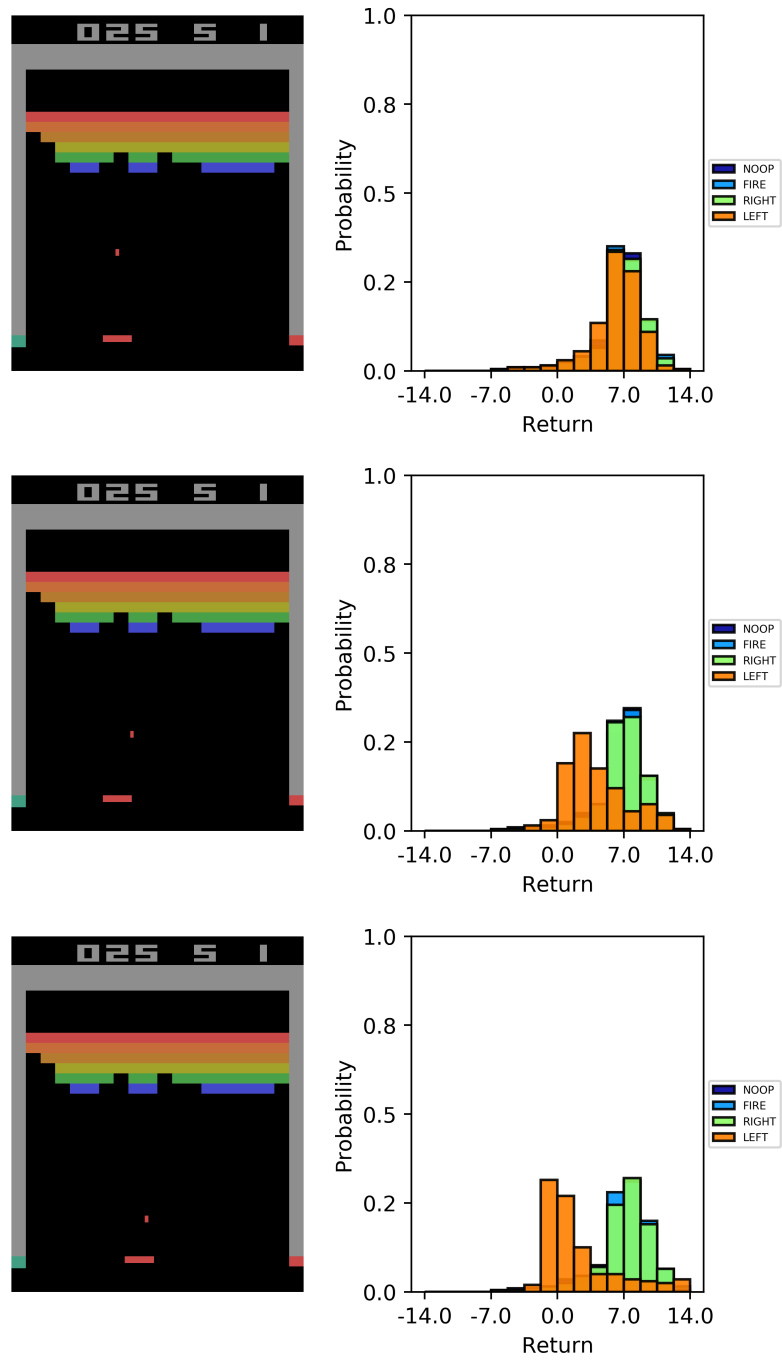


Figure 6: This example shows 3 consecutive frames and the approximate return distributions for all the actions in the Breakout game played by our MMDQN. The approximate return distributions plotted here are the histograms with 17 bins constructed from the learnt particles by MMDQN.